Advance Pipelines and Remoting

23 March 2026
10:20

WHERE-OBJECT Operator

Here operator requires script block. Whatever conditions are written in script block, it matches with get-service command and gives the output.
Get-service | where-object {$_.Status -eq "running" -and $_.name -like "*win*"}

Script Block of where object

```
PS C:\Users\lenovo> Get-service | where-object {$_.Status -eq "running" -and $_.name -like "*win*"}

Status   Name               DisplayName
------   ----               -----------
Running  WinDefend          Microsoft Defender Antivirus Service
Running  WinHttpAutoProx... WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt            Windows Management Instrumentation
```

Note: Make sure you always filter an the object on the left side of pipeline(recommended by microsoft). Here in the previous example it is filtered on right side should be avoided.
Avoid using where object. Filter it by using name, status parameter.
Example:
Get-service | select-object Name ,status -first 10

```
PS C:\Users\lenovo> Get-service | select-object Name ,status -first 10
>>

Name             Status
----             ------
AarSvc_63b92f5   Stopped
AdobeARMservice  Running
AJRouter         Stopped
ALG              Stopped
AppIDSvc         Stopped
Appinfo          Running
AppMgmt          Stopped
AppReadiness     Stopped
AppVClient       Stopped
AppXSvc          Running
```

For -Each Object

Get-service | foreach-object {write-host " The Service name is $($PSItem.name)"}
This command Is used to list all the service names

```
PS C:\Users\lenovo> Get-service | foreach-object {write-host " The Service name is $($PSItem.name)"}
 The Service name is AarSvc_63b92f5
 The Service name is AdobeARMservice
 The Service name is AJRouter
 The Service name is ALG
 The Service name is AppIDSvc
 The Service name is Appinfo
 The Service name is AppMgmt
 The Service name is AppReadiness
 The Service name is AppVClient
 The Service name is AppXSvc
 The Service name is AssignedAccessManagerSvc
 The Service name is AudioEndpointBuilder
 The Service name is Audiosrv
 The Service name is autotimesvc
 The Service name is AxInstSV
 The Service name is BcastDVRUserService_63b92f5
 The Service name is BDESVC
 The Service name is BFE
 The Service name is BITS
 The Service name is BluetoothUserService_63b92f5
 The Service name is BrokerInfrastructure
 The Service name is BTAGService
 The Service name is BthAvctpSvc
 The Service name is bthserv
 The Service name is camsvc
 The Service name is CaptureService_63b92f5
 The Service name is cbdhsvc_63b92f5
 The Service name is CDPSvc
 The Service name is CDPUserSvc_63b92f5
 The Service name is CertPropSvc
 The Service name is ClipSVC
 The Service name is cloudidsvc
 The Service name is COMSysApp
 The Service name is ConsentUxUserSvc_63b92f5
 The Service name is CoreMessagingRegistrar
 The Service name is cphs
```

Types of Pipeline Inputs

Get-help Get-service -full
Search for this

```
-Name <string[]>

    Required?                    false
    Position?                    0
    Accept pipeline input?       true (ByValue, ByPropertyName)
    Parameter set name           Default
    Aliases                      ServiceName
    Dynamic?                     false
```

Here this is -Name parameter which excepts String and also excepts input from the pipeline I,e BypropertyName or ByValue
@("Bits","winrm") | get-service
Here a dummy array (@("Bits","winrm")) is created. The get-service will get filter out the bits and winrm and provide me the output.

```
PS C:\Users\lenovo> @("Bits","winrm") | get-service
>>

Status   Name               DisplayName
------   ----               -----------
Stopped  Bits               Background Intelligent Transfer Ser...
Stopped  winrm              Windows Remote Management (WS-Manag...
```

@(1,2) | get-service

In this example there is not match

$a =(
@{
Name = "Amar"
},
@{
Name="power"
}
)
To check - **$a**

$b = $a | select-object @{l = "name" ; e={$_.values}}
$b : to check

```
PS C:\Users\lenovo> $a =(
>> @{
>> Name = "Amar"
>> },
>> @{
>> Name="power"
>> }
>> )
>>
PS C:\Users\lenovo> $a

Name                           Value
----                           -----
Name                           Amar
Name                           power


PS C:\Users\lenovo> $b =$a | select-object @{l="name"; e={$_.Values}}
PS C:\Users\lenovo> $b

name
----
Amar
power
```

$b | get-process
Get-process | select-object processname -first 10

```
PS C:\Users\lenovo> Get-process | select-object processname -first 10
>>
>>

ProcessName
-----------
AggregatorHost
ApplicationFrameHost
armsvc
audiodg
chrome
chrome
chrome
chrome
chrome
chrome
```

Here we check the name of process running.

$a =(
@{
Name = "Chrome"
},
@{
Name="powershell"
}
)
The name in the input should match the parameter.

```
PS C:\Users\lenovo> $a =(
>> @{
>> Name = "Chrome"
>> },
>> @{
>> Name="powershell"
>> }
>> )
>>
>>
PS C:\Users\lenovo> $a

Name                           Value
----                           -----
Name                           Chrome
Name                           powershell


PS C:\Users\lenovo> $b =$a | select-object @{l="name"; e={$_.Values}}
PS C:\Users\lenovo> $b

name
----
Chrome
powershell


PS C:\Users\lenovo> $b | get-process

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
    219      19    22476      48544       3.13   2168   8 chrome
    241      14    13724      20276       0.06   5092   8 chrome
    184      13    10704      19860       0.06   6080   8 chrome
    192      17    20432      30236       0.14   7080   8 chrome
   1653      64    75984     188088      33.20   7528   8 chrome
    224      20    26100      65144       0.27   7624   8 chrome
    465      22    23116      31892       0.22   7772   8 chrome
    322      29   109696     163004      16.88   9340   8 chrome
    369      22    20772      45320       4.55  11624   8 chrome
    231      10     7040       9512       0.05  13176   8 chrome
    567      29    68000      88216      14.95  13228   8 chrome
    254      23    47152      91140       1.28  13260   8 chrome
    220      19    18472      46872       0.14  13304   8 chrome
    741      34    63988      82696       3.19   7020   8 powershell


PS C:\Users\lenovo>
```

Prepare a custom function used to show time

Function show-uptime {
Param(
[Parameter(ValueFromPipeline)]
[string []] $computername=$Env:COMPUTERNAME
)
Foreach($c in $computename){
Write-host "The Computer-name is $c" -foregrroundcolor "yellow"
}
}

To check **- show-uptime**

```
PS C:\Users\lenovo> Function show-uptime {
>> Param(
>> [Parameter(ValueFromPipeline)]
>> [string []] $computername=$Env:COMPUTERNAME
>> )
>> Foreach($c in $computename){
>> Write-host "The Computer-name is $c" -foregrroundcolor "yellow"
>> }
>> }
>>
PS C:\Users\lenovo> @("DC1" , "WIN10")_|Show-uptime
PS C:\Users\lenovo> Show-uptime
PS C:\Users\lenovo>
```

Using begin, process, end

Function show-uptime {
Param(
[Parameter(ValueFromPipeline)]
[string []] $computername=$Env:COMPUTERNAME
)
Begin{
Write-host "Begin block called" - foregroundcolor "Green"
}
Process{
Write-host "The Computer-name is $c" -foregrroundcolor "yellow"
}
End{
Write-host "end Block called"
}
}
@("DCI","WIN10") | show-uptime - to check

```
PS C:\Users\lenovo> Function show-uptime {
>> Param(
>> [Parameter(ValueFromPipeline)]
>> [string []] $computername=$Env:COMPUTERNAME
>> )
>> Begin{
>> Write-host "Begin block called" - foregroundcolor "Green"
>> }
>> Process{
>> Write-host "The Computer-name is $c" -foregrroundcolor "yellow"
>> }
>> End{
>> Write-host "end Block called"
>> }
>> }
PS C:\Users\lenovo> @("DC1" , "WIN10") |Show-uptime
Begin block called - foregroundcolor Green
The Computer-name is  -foregrroundcolor yellow
The Computer-name is  -foregrroundcolor yellow
end Block called
PS C:\Users\lenovo>
```

Get-help show-uptime -full

```
PS C:\Users\lenovo> Get-help show-uptime -full
>>

NAME
    show-uptime

SYNTAX
    show-uptime [[-computername] <string[]>]  [<CommonParameters>]


PARAMETERS
    -computername <string[]>

        Required?                    false
        Position?                    0
        Accept pipeline input?       true (ByValue)
        Parameter set name           (All)
        Aliases                      None
        Dynamic?                     false

    <CommonParameters>
        This cmdlet supports the common parameters: Verbose, Debug,
        ErrorAction, ErrorVariable, WarningAction, WarningVariable,
        OutBuffer, PipelineVariable, and OutVariable. For more information, see
        about_CommonParameters (https://go.microsoft.com/fwlink/?LinkID=113216).


INPUTS
    System.String[]


OUTPUTS
    System.Object

ALIASES
    None


REMARKS
    None
```

$a =@(
@{
Name ="DC1"
},
@{
Name="WIN10"
}
)
$b =$a | select-object @{l="computername"; e={$_.Values}}
To check
$b | show-uptime

```
PS C:\Users\lenovo> $a =@(
>> @{
>> Name ="DC1"
>> },
>> @{
>> Name="WIN10"
>> }
>> )
>> $b =$a | select-object @{l="computer-name"; e={$_.Values}}
>>
PS C:\Users\lenovo> $b |show-uptime
Begin block called - foregroundcolor Green
The Computer-name is  -foregrroundcolor yellow
The Computer-name is  -foregrroundcolor yellow
end Block called
```

(I am getting wrong output)
If we use MachineName rather then computername - it Doesn't work

$a =@(
@{
Name ="DC1"
},
@{
Name="WIN10"
}
)
$b =$a | select-object @{l="MachineName"; e={$_.Values}}

Example
Function show-uptime {
Param(
[Parameter(ValueFromPipeline)]
[string []] $computername=$Env:COMPUTERNAME
)
Process{
Get-CimInstance -computerName $computerName -className

```
Win32_operatingsystem | select-object LastBootUpTime

}
}
```